DAYANANDA SAGAR COLLEGE OF ENGINEERING
(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi, Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560 111, India

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING
(Accredited by NBA Tier 1: 2025-2028)

Introduction to Deep Learning (22IS741)
AAT report
on

AAT1: Perform Basic Tensor Operations using TensorFlow
AAT2: Driver Drowsiness Detection using CNN

Bachelor of Engineering in
Information Science and Engineering

Submitted by

ARPIT SRIVASTAVA	(1DS23IS023)
KHUSHI AGRAWAL	(1DS23IS074)
HIMANSHU	(1DS23IS061)
HARSH KUMAR GUPTA	(1DS23IS054)

Under the Guidance of

Dr. Rajeshwari J - Associate Professor
Prof Ayush Mishra - Assistant Professor
Dept of ISE

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATAKA, INDIA
2026-27

TABLE OF CONTENTS

Sl. no.	Topics	Page no.

Perform Basic Tensor Operations using TensorFlow

1	Introduction	1
2	Objectives	1
3	Case Description / Background Information	2 - 3
4	Analysis and Discussion	3 - 5
5	Results	5 - 7
6	Conclusion	7

Driver Drowsiness Detection using CNN

1	Introduction	8
2	Objectives	8
3	Methodology	8 - 12
4	Results (With Screenshots)	12 - 16
5	Software Used	16 - 17
6	Conclusion	17

Perform Basic Tensor Operations using TensorFlow

1. Introduction

This experiment creates tensors of rank 0 to 3 in TensorFlow 2.20 and applies eleven basic operations to them: addition, subtraction, element-wise multiplication, matrix multiplication, division, transpose, reshape, square and three reductions. Every result was checked by hand and matches the expected value.

A tensor is a multi-dimensional array of numbers and is the basic data structure of every deep learning framework. An image is a 3D tensor (height, width, channels), a batch of images is a 4D tensor, and the weights of a layer are a 2D tensor. Training a neural network is, at its core, a long chain of tensor operations: the forward pass multiplies inputs by weights and adds a bias, and the backward pass computes gradients of these same operations.

TensorFlow stores data as tensors instead of plain Python lists because tensors can run on a GPU, support automatic differentiation and apply one operation to every element at once (vectorisation). Learning these operations first makes the layers of a CNN, used in AAT 2, much easier to understand.

- Element-wise operation: Tensor a, Tensor b → same position combined → Tensor of the same shape
- Matrix multiplication: rows of a × columns of b → dot products → new matrix
- Reduction: Tensor → sum, mean or maximum along an axis → smaller tensor or a single value

2. Objectives

The aim is to create tensors in TensorFlow and perform the basic operations on which neural networks are built. The experiment has five objectives:

1. Create scalar, vector, matrix and 3D tensors, and special tensors (zeros, ones and random values).
2. Perform element-wise arithmetic: addition, subtraction, multiplication and division.
3. Perform matrix operations: matrix multiplication, transpose, reshape and element-wise square.
4. Apply reduction operations: sum, mean, maximum and argmax.
5. Compare immutable constants with mutable variables, which hold the weights of a model.

Expected outcomes

1. A working notebook in which every operation prints its result, shape and data type.
2. A clear difference between element-wise multiplication and matrix multiplication.
3. An understanding of how shape and data type change after each operation.
4. A link between these operations and the layers of a neural network.

3. Case Description / Background Information

The case uses two small 2×2 integer matrices, a = [[1, 2], [3, 4]] and b = [[5, 6], [7, 8]]. Small values were chosen on purpose so that every output can be verified by hand. The same operations work unchanged on tensors with millions of elements.

Experiment overview

Attribute	Value
Framework	TensorFlow 2.20.0 (Python 3)
Input tensors	a = [[1, 2], [3, 4]], b = [[5, 6], [7, 8]]
Shape of a and b	(2, 2)
Data type	int32 (float64 after division)
Operations applied	11, plus tensor creation
Environment	Jupyter notebook

Key concepts

- **Rank:** the number of dimensions of a tensor. A scalar has rank 0, a vector rank 1, a matrix rank 2.
- **Shape:** the number of elements along each dimension, for example (2, 2) for a 2×2 matrix.
- **Data type (dtype):** the type of every element, such as int32 or float32. All elements share one dtype.
- **Element-wise operation:** combines the elements in the same position of two tensors of the same shape.
- **Reduction:** collapses one or more dimensions into a single summary value.

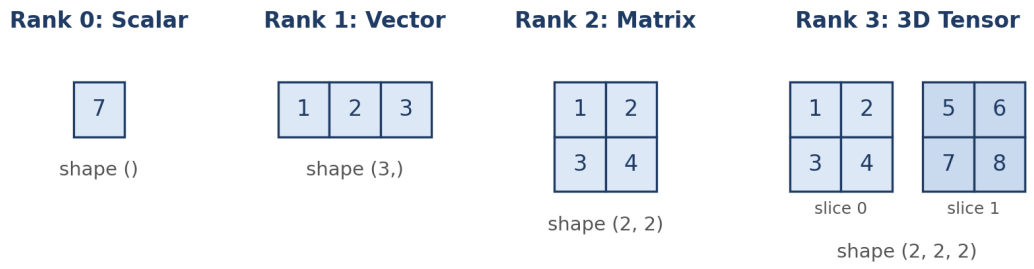


Fig 1: Tensors of rank 0 to 3 used in the experiment

Operations used

Group	TensorFlow function	What it does
Creation	tf.constant, tf.zeros, tf.ones, tf.random.uniform	Builds a tensor from values, or filled with 0, 1 or random numbers
Element-wise	tf.add, tf.subtract, tf.multiply, tf.divide	Combines elements in the same position
Matrix	tf.matmul	Row by column dot product, as in a dense layer

Group	TensorFlow function	What it does
Shape	tf.transpose, tf.reshape	Swaps rows and columns, or gives the data a new shape
Power	tf.square	Squares every element
Reduction	tf.reduce_sum, tf.reduce_mean, tf.reduce_max, tf.argmax	Summarises a tensor or finds the position of the maximum
State	tf.constant, tf.Variable	Fixed value versus a value that can be updated

4. Analysis and Discussion

The notebook follows a six-stage workflow. Each stage builds on the previous one, starting with tensor creation and ending with variables, which are the trainable part of every model.

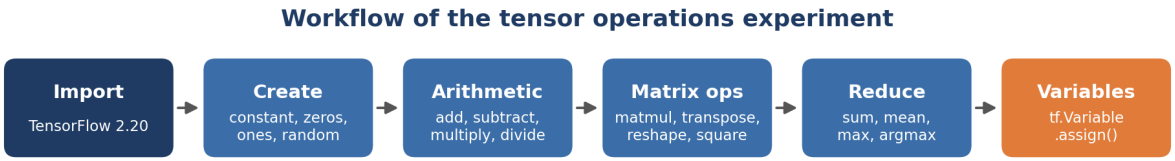


Fig 2: Workflow of the experiment

Implementation

1. Import TensorFlow and print its version (2.20.0).
2. Create a scalar, a vector, a matrix and a 3D tensor with tf.constant, and check their shapes.
3. Create special tensors with tf.zeros, tf.ones and tf.random.uniform.
4. Define the two matrices a and b.
5. Apply tf.add, tf.subtract, tf.multiply and tf.divide.
6. Apply tf.matmul, tf.transpose, tf.reshape and tf.square.
7. Apply tf.reduce_sum, tf.reduce_mean, tf.reduce_max and tf.argmax.
8. Create a constant and a variable, and update the variable with assign().

```
import tensorflow as tf
a = tf.constant([[1, 2], [3, 4]])
b = tf.constant([[5, 6], [7, 8]])

tf.add(a, b)            # element-wise sum
tf.multiply(a, b)       # element-wise product
tf.matmul(a, b)         # matrix product (rows x columns)
tf.reshape(a, [4, 1])   # same 4 values, new shape
tf.reduce_sum(a)        # 10
```

Element-wise multiplication versus matrix multiplication

The two kinds of multiplication are the most common source of confusion. tf.multiply pairs each element with the element in the same position, so both tensors must have the same shape. tf.matmul takes the dot product of a row of the first matrix with a column of the second, so the number of columns of a must equal the number of rows of b. A dense neural network layer computes tf.matmul(x, W) + b.

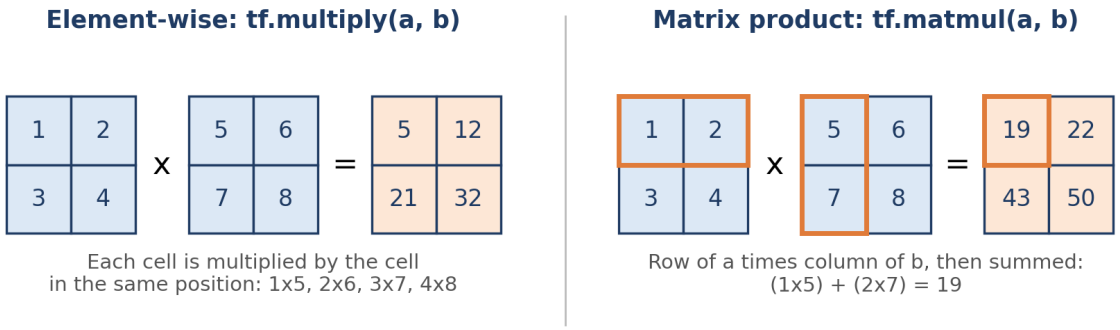


Fig 3: Element-wise multiplication compared with matrix multiplication

tf.multiply (element-wise)	tf.matmul (matrix product)
Shapes must match (or broadcast)	Inner dimensions must match
1×5, 2×6, 3×7, 4×8	1×5 + 2×7 = 19, and so on
Result [[5, 12], [21, 32]]	Result [[19, 22], [43, 50]]
Commutative: a·b = b·a	Not commutative: ab ≠ ba in general
Used for masks and scaling	Used in dense and attention layers

Shape, data type and reduction

Shape changes do not change the data. tf.transpose swaps rows and columns, and tf.reshape lays the same four values out as a 4×1 column, which is only allowed because 2×2 and 4×1 both hold four elements. Data types do change in one place: dividing two int32 tensors returns float64, because the result of a division is not always a whole number. Reductions remove dimensions. Note that tf.reduce_mean on an int32 tensor returns 2 instead of 2.5, because integer arithmetic drops the fraction, so the tensor should be cast to float before taking a mean.

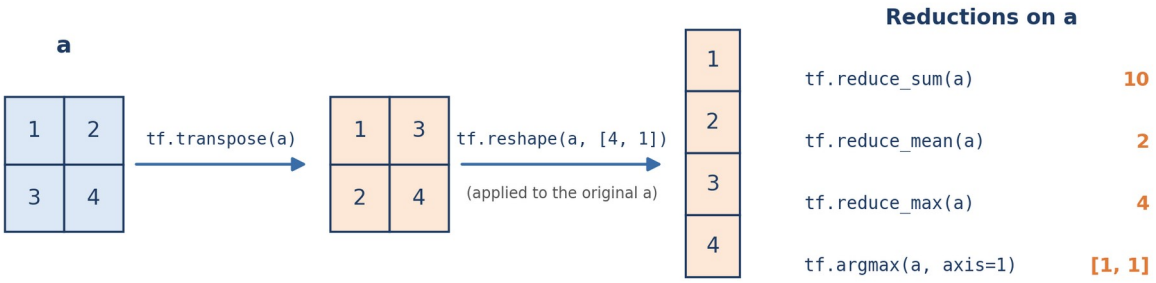


Fig 4: Transpose, reshape and reduction operations on tensor a

Constants versus variables

tf.constant	tf.Variable
Value is fixed after creation	Value can be changed with assign()
Used for input data and fixed values	Used for weights and biases
Calling assign() raises an error	assign(20) changed the value from 10 to 20
Not tracked for training	Tracked by the optimiser during training

Strengths and limitations of the approach

Strengths	Limitations
Small inputs make every result easy to verify by hand	2×2 tensors do not show the speed benefit of a GPU
Covers creation, arithmetic, shape, reduction and state	Broadcasting between different shapes is not tested
Each cell prints shape and dtype as well as values	Integer tensors hide fractions in mean and division

5. Results

All operations ran without errors and every output matched the value calculated by hand. The screenshots below are taken from the executed notebook.

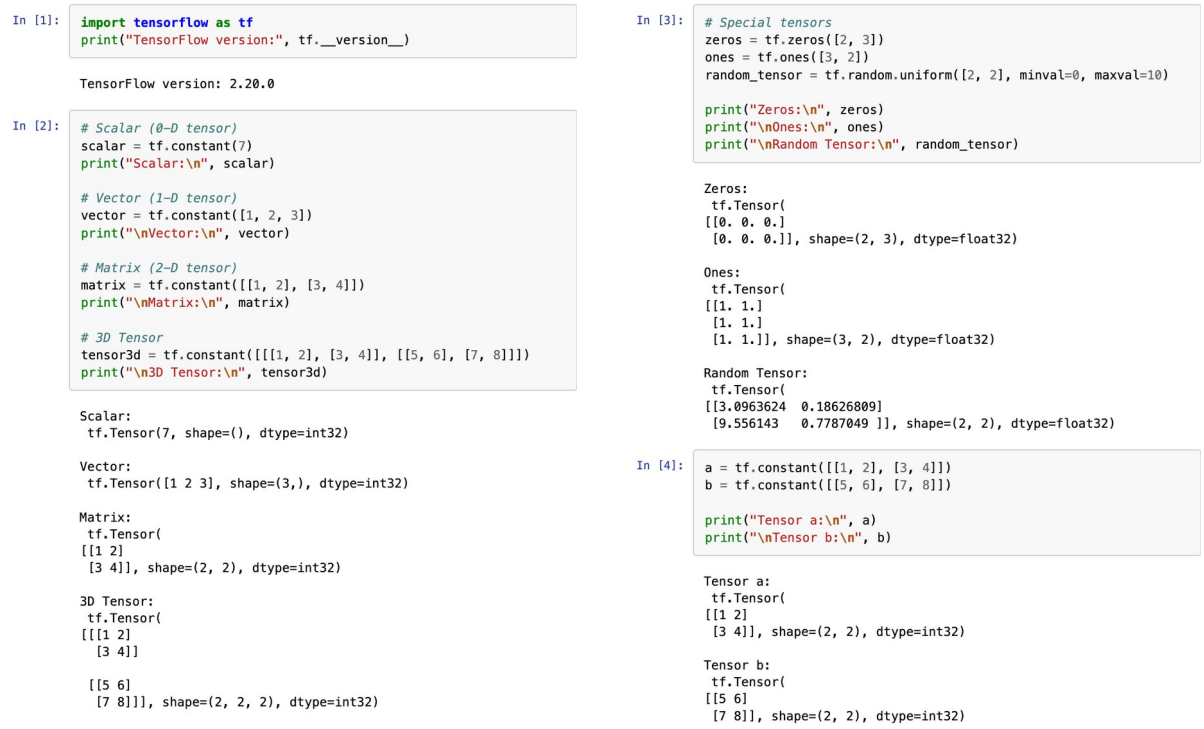


Fig 5: Creating tensors, special tensors and the input matrices a and b

```
In [5]:
add_result = tf.add(a, b)
print("a + b =\n", add_result)

a + b =
 tf.Tensor(
[[ 6  8]
 [10 12]], shape=(2, 2), dtype=int32)

In [6]:
sub_result = tf.subtract(a, b)
print("a - b =\n", sub_result)

a - b =
 tf.Tensor(
[[-4 -4]
 [-4 -4]], shape=(2, 2), dtype=int32)

In [7]:
mul_result = tf.multiply(a, b)
print("a * b (element-wise) =\n", mul_result)

a * b (element-wise) =
 tf.Tensor(
[[ 5 12]
 [21 32]], shape=(2, 2), dtype=int32)

In [8]:
matmul_result = tf.matmul(a, b)
print("a @ b (matrix multiplication) =\n", matmul_result)

a @ b (matrix multiplication) =
 tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)

In [9]:
div_result = tf.divide(a, b)
print("a / b =\n", div_result)

a / b =
 tf.Tensor(
[[0.2        0.33333333]
 [0.42857143 0.5       ]], shape=(2, 2), dtype=float64)
```

Fig 6: Element-wise operations, matrix multiplication and division

```
In [10]:
transpose_result = tf.transpose(a)
print("Transpose of a =\n", transpose_result)

Transpose of a =
 tf.Tensor(
[[1 3]
 [2 4]], shape=(2, 2), dtype=int32)

In [11]:
reshape_result = tf.reshape(a, [4, 1])
print("Reshaped a (4x1) =\n", reshape_result)

Reshaped a (4x1) =
 tf.Tensor(
[[1]
 [2]
 [3]
 [4]], shape=(4, 1), dtype=int32)

In [13]:
square_result = tf.square(a)
print("Square of a (element-wise) =\n", square_result)

Square of a (element-wise) =
 tf.Tensor(
[[ 1  4]
 [ 9 16]], shape=(2, 2), dtype=int32)

In [12]:
print("Sum of all elements in a:", tf.reduce_sum(a).numpy())
print("Mean of all elements in a:", tf.reduce_mean(a).numpy())
print("Max value in a:", tf.reduce_max(a).numpy())
print("Index of max value in each row of a:", tf.argmax(a, axis=
1).numpy())

Sum of all elements in a: 10
Mean of all elements in a: 2
Max value in a: 4
Index of max value in each row of a: [1 1]

In [14]:
const_val = tf.constant(10)
var_val = tf.Variable(10)

var_val.assign(20)  # allowed
print("Constant:", const_val.numpy())
print("Updated Variable:", var_val.numpy())

# const_val.assign(20)  # This would raise an error, constants c
an't be reassigned

Constant: 10
Updated Variable: 20
```

Fig 7: Transpose, reshape, square, reductions, and constants versus variables

Summary of results

Operation	Result	Shape	Dtype
tf.add(a, b)	[[6, 8], [10, 12]]	(2, 2)	int32
tf.subtract(a, b)	[[-4, -4], [-4, -4]]	(2, 2)	int32
tf.multiply(a, b)	[[5, 12], [21, 32]]	(2, 2)	int32
tf.matmul(a, b)	[[19, 22], [43, 50]]	(2, 2)	int32
tf.divide(a, b)	[[0.2, 0.333], [0.429, 0.5]]	(2, 2)	float64
tf.transpose(a)	[[1, 3], [2, 4]]	(2, 2)	int32
tf.reshape(a, [4, 1])	[[1], [2], [3], [4]]	(4, 1)	int32
tf.square(a)	[[1, 4], [9, 16]]	(2, 2)	int32
tf.reduce_sum(a)	10	()	int32
tf.reduce_mean(a)	2	()	int32
tf.reduce_max(a)	4	()	int32
tf.argmax(a, axis=1)	[1, 1]	(2,)	int64

Discussion of the results

- **Element-wise results are correct:** each output cell depends only on the two input cells in the same position.
- **Matrix product differs from element-wise product:** [[19, 22], [43, 50]] against [[5, 12], [21, 32]], which confirms that the two operations must not be mixed up.
- **Division changes the data type:** int32 inputs give a float64 output.
- **Integer mean is truncated:** the true mean of a is 2.5, but the int32 result is 2.
- **Variables are mutable:** assign(20) changed the variable, while the constant stayed at 10.

6. Conclusion

TensorFlow tensors of rank 0 to 3 were created, and eleven basic operations were applied and verified by hand. The project achieved the following:

- Created scalar, vector, matrix, 3D and special tensors and checked their shapes and data types.
- Performed element-wise addition, subtraction, multiplication and division.
- Showed that tf.matmul and tf.multiply give different results on the same inputs.
- Applied transpose, reshape, square and the sum, mean, max and argmax reductions.
- Showed that tf.Variable can be updated while tf.constant cannot.

These operations are the building blocks of every layer in a neural network: a dense layer is a matrix multiplication followed by an addition, a convolution is a sliding element-wise multiplication followed by a sum, and training updates tf.Variable weights. The CNN in AAT 2 is built entirely from these operations. Future work could cover broadcasting, operations on GPU with large tensors, and automatic differentiation with tf.GradientTape.

Driver Drowsiness Detection using CNN

1. Introduction

DriveAlert is a deep learning system that watches a driver through a camera, uses a Convolutional Neural Network to decide in every frame whether the driver's eyes are open or closed, and raises an alarm when the eyes stay closed for 15 frames in a row. The eye-state CNN reaches 99.45% accuracy on 6,319 held-out eye images and 90.22% on 3,486 real eye photographs from a completely different source.

Drowsy driving is a major cause of road accidents, especially on highways and at night. A tired driver reacts slowly, drifts out of the lane and can fall asleep for a few seconds without noticing. Unlike alcohol, drowsiness cannot be measured with a simple test at the roadside, so it has to be detected while the person is driving.

The most reliable visible sign of drowsiness is long eye closure: an alert person blinks in about a tenth of a second, while a drowsy person keeps the eyes closed for much longer. DriveAlert therefore splits the problem into two steps. A CNN classifies each eye as Open or Closed, and a simple frame counter turns those per-frame decisions into a drowsiness alert. The system runs on an ordinary laptop webcam and is also available as a Gradio web application.

The system is a driver assistance tool and is not a replacement for adequate rest before driving.

2. Objectives

The main objective is to build a camera-based system that detects driver drowsiness in real time using a CNN. The project has the following objectives:

1. **Detect eye state:** classify each eye as Open or Closed with a CNN trained from scratch.
2. **Prepare the data:** extract eye regions from face images, normalise them and use augmentation to improve generalisation.
3. **Evaluate honestly:** split data by face, and test on real eye photographs from a different source.
4. **Detect drowsiness:** raise an alert when the eyes stay closed for 15 consecutive frames.
5. **Explain the model:** use Grad-CAM to show which image regions the CNN uses.
6. **Provide an interface:** a Gradio web application and a real-time webcam monitor.

Expected outcomes

- A trained CNN that separates open and closed eyes.
- Training curves, confusion matrices and ROC curves that show how well the model learned.
- A working application that marks a driver as alert or drowsy.
- A clear explanation of why an eye-level model is used instead of a whole-face model.

3. Methodology

3.1 Overview of the proposed system

DriveAlert processes every video frame in six stages. The face is found with a Haar cascade detector, two square eye regions are cut out using the geometry of the face, the CNN gives the probability that the eyes are closed, and a counter decides when to alert the driver.

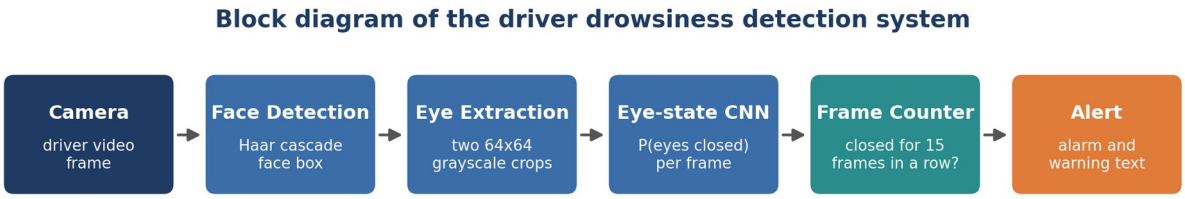


Fig 1: Block diagram of the driver drowsiness detection system

3.2 Design choice: whole face or eye state

The first design classified the whole face directly as Drowsy or Non Drowsy using the public Driver Drowsiness Dataset (41,793 face images of 28 people). To test it fairly, the data was split by person, so the test faces were never seen in training. The CNN reached 98.02% accuracy on training faces but only 69.67% on the unseen people (AUC 0.58), barely above the 58.50% obtained by always answering Drowsy, and a pre-trained MobileNetV2 feature extractor with a linear classifier reached only 49.70%. With few people, a whole-face model learns who the person is instead of how tired they look, and the dataset labels a whole video clip as drowsy even when many frames show open eyes.

Approach	Input	Accuracy on unseen faces	Decision
CNN on whole face	96×96 colour face	69.67%	Rejected
MobileNetV2 features on whole face	128×128 colour face	49.70%	Rejected
CNN on eye state (final)	64×64 grayscale eye	99.45%	Selected

The eye-state design was selected because eye closure is a clear, frame-level visual signal that does not depend on the identity of the driver.

3.3 Dataset and eye extraction

Training data comes from the open and closed eyes dataset on Hugging Face, which contains face images with a bounding box for each eye and a label for the eye state. A sample of 21,099 faces was used. Each eye was cut out as a square of 1.6 times the eye width, so the crop size does not depend on how open the eye is, and resized to 64×64 grayscale. Both eyes of one face are always kept in the same split. A separate set of 3,486 real eye photographs from a different source was kept only for testing.

Attribute	Value
Source faces	21,099
Eye crops	42,131
Train / validation / test	29,495 / 6,317 / 6,319
Real-world test eyes	3,486 (1,554 closed, 1,932 open)
Input size	64 × 64 × 1 (grayscale)
Classes	Open (0), Closed (1)



Fig 2: Extraction of the two eye regions from a face image

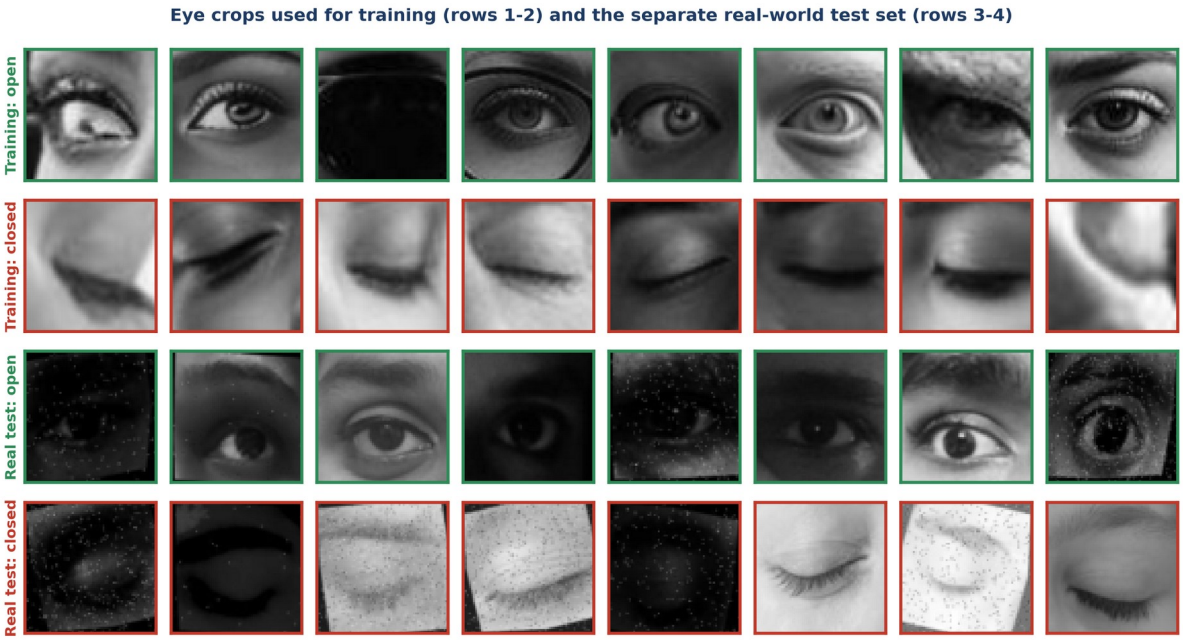


Fig 3: Sample eye crops from the training data and the real-world test set

3.4 Preprocessing and data augmentation

Pixel values are scaled from 0 to 255 to the range 0 to 1. Augmentation is applied only to training images, so that the CNN sees varied versions of every eye. Blur and noise are applied to both classes, so that image sharpness cannot be used as a shortcut to the label.

Setting	Value	Purpose
Rescale	1/255	Normalises pixel values
Horizontal flip	Yes	Left and right eyes look alike
Rotation	±8%	Handles head tilt
Zoom / shift	15% / 8%	Handles differences in crop position
Contrast / brightness	30% / 20%	Handles lighting changes in a car
Blur (3×3 average)	35% of images	Handles motion and out-of-focus frames

Setting	Value	Purpose
Gaussian noise (σ = 0.05)	35% of images	Handles low-light camera noise

3.5 Model architecture

The CNN is built from scratch with three convolution blocks. Each block has two 3×3 convolutions with batch normalisation and ReLU, followed by max pooling and dropout. Global average pooling and two dense layers turn the features into one probability. The model has 304,417 parameters, which is small enough to run in real time on a laptop CPU.

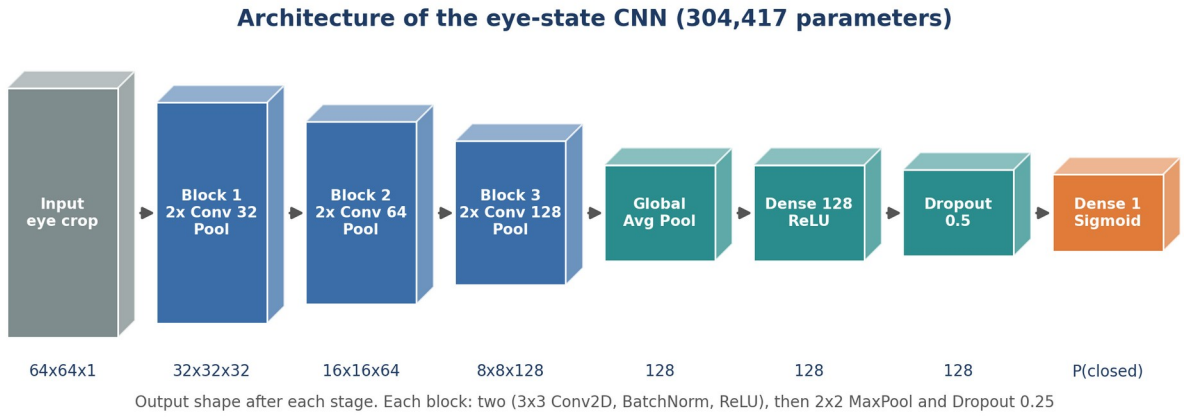


Fig 4: Architecture of the eye-state CNN

Layer	Setting	Output shape
Input	grayscale eye crop	64×64×1
Conv block 1	2 × Conv 32, BN, ReLU, MaxPool, Dropout 0.25	32×32×32
Conv block 2	2 × Conv 64, BN, ReLU, MaxPool, Dropout 0.25	16×16×64
Conv block 3	2 × Conv 128, BN, ReLU, MaxPool, Dropout 0.25	8×8×128
GlobalAveragePooling2D	one value per feature map	128
Dense	128 units, ReLU	128
Dropout	0.5	128
Dense (output)	1 unit, sigmoid	P(closed)

3.6 Training

The model is trained with the Adam optimiser (learning rate 0.001) and binary cross-entropy loss, with a batch size of 128. The learning rate is halved when the validation loss stops improving for 2 epochs, and training stops early after 4 epochs without improvement, keeping the best weights. Training ran for 12 epochs on an Apple M1 Pro CPU with TensorFlow 2.20. The batch normalisation momentum was set to 0.9 instead of the default 0.99, because with the default value the stored statistics had not caught up after the first epochs and the model predicted Closed for every image at inference time.

3.7 Drowsiness decision

A single closed-eye frame is normal, because people blink. The system averages P(closed) of the two eyes and counts consecutive frames with a value of at least 0.5. When 15 frames in a row are closed, which is about half a second at 30 frames per second and longer than a normal blink, the alarm sounds and a DROWSINESS ALERT message is shown. Any open-eye frame resets the counter.

3.8 System workflow

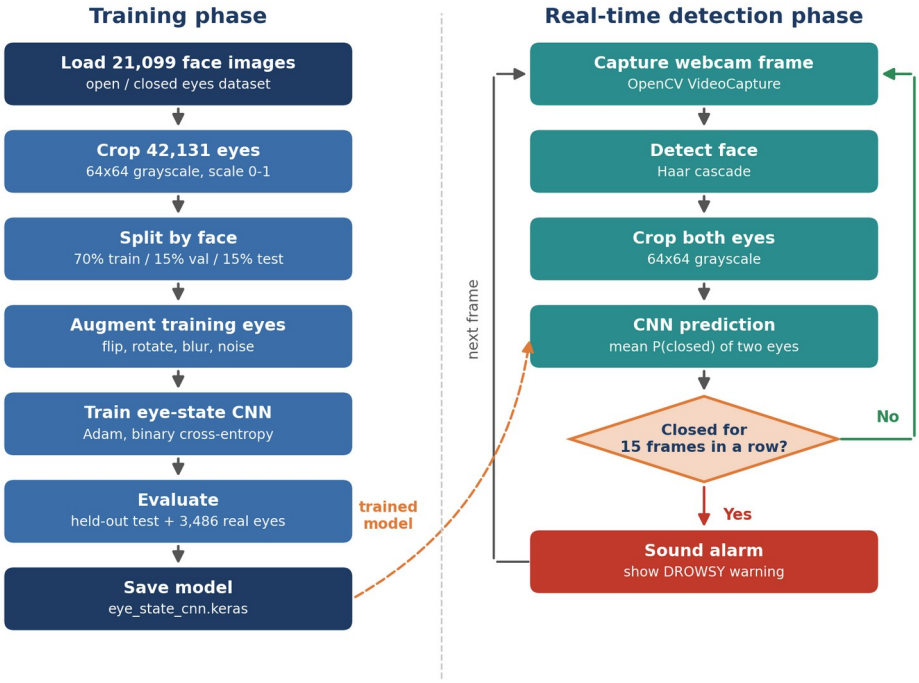


Fig 5: Training and real-time detection workflow

1. **Collect data:** load face images with eye boxes and eye-state labels.
2. **Extract eyes:** crop both eyes as 64×64 grayscale images.
3. **Split:** 70% training, 15% validation and 15% test, keeping both eyes of a face together.
4. **Augment:** apply flip, rotation, zoom, contrast, brightness, blur and noise to training eyes.
5. **Train:** fit the CNN with Adam and binary cross-entropy, with early stopping.
6. **Evaluate:** measure accuracy, precision, recall, F1 and AUC on the test set and the real-world set.
7. **Deploy:** load the model in the Gradio application and the webcam monitor.
8. **Detect:** find the face, crop both eyes and predict P(closed) in every frame.
9. **Alert:** sound the alarm after 15 consecutive closed-eye frames.

4. Results (With Screenshots)

The figures below show the training behaviour, the evaluation of the trained CNN, a system-level test on real driver videos and the working application.

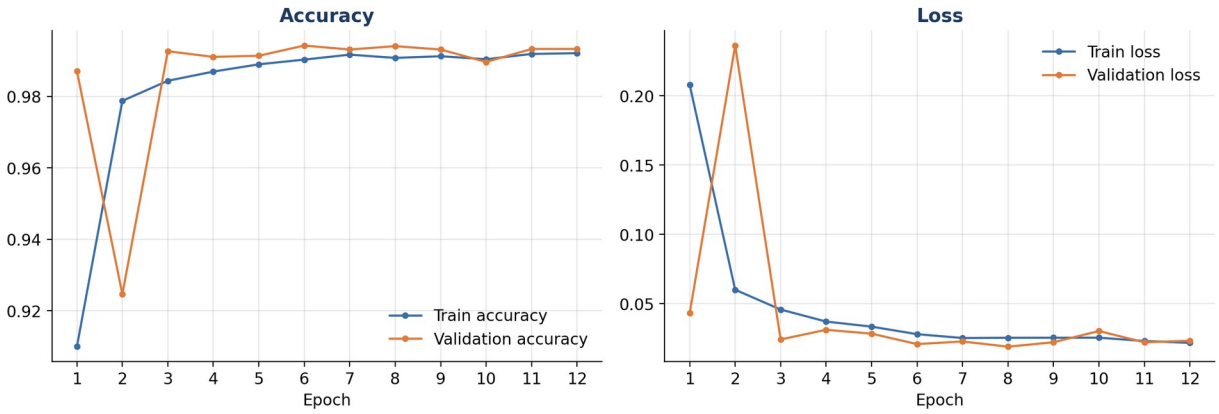


Fig 6: Training and validation accuracy and loss

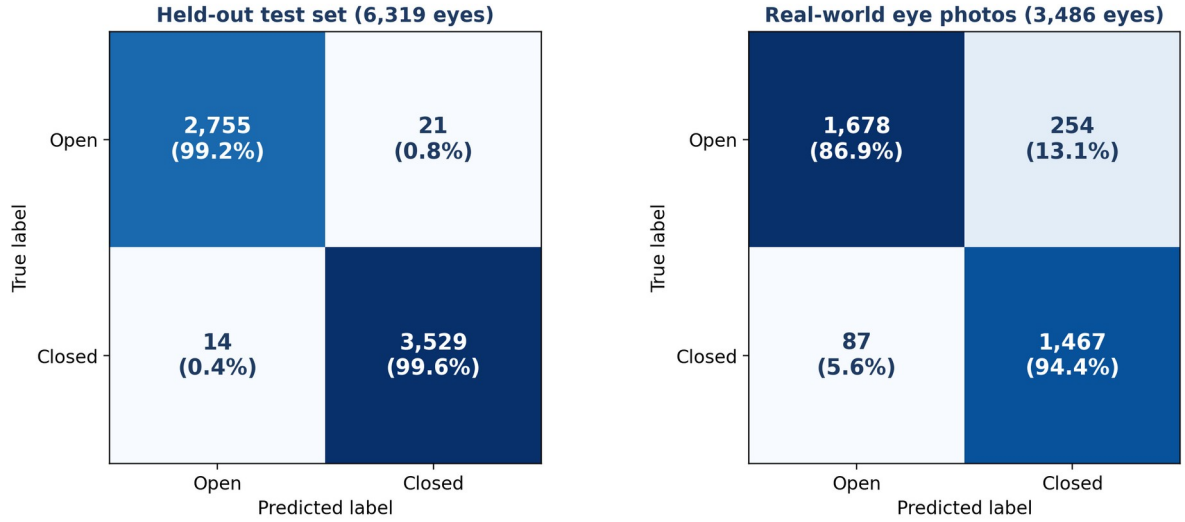


Fig 7: Confusion matrices on the held-out test set and the real-world set

Metrics from the confusion matrices

Measure	Calculation	Held-out test	Real-world
True Positive (TP)	Closed predicted as Closed	3,529	1,467
True Negative (TN)	Open predicted as Open	2,755	1,678
False Positive (FP)	Open predicted as Closed	21	254
False Negative (FN)	Closed predicted as Open	14	87
Accuracy	(TP + TN) / total	0.9945	0.9022
Precision	TP / (TP + FP)	0.9941	0.8524
Recall	TP / (TP + FN)	0.9960	0.9440
F1-score	2 × P × R / (P + R)	0.9951	0.8959
Specificity	TN / (TN + FP)	0.9924	0.8685
ROC AUC	area under the ROC curve	0.9999	0.9623

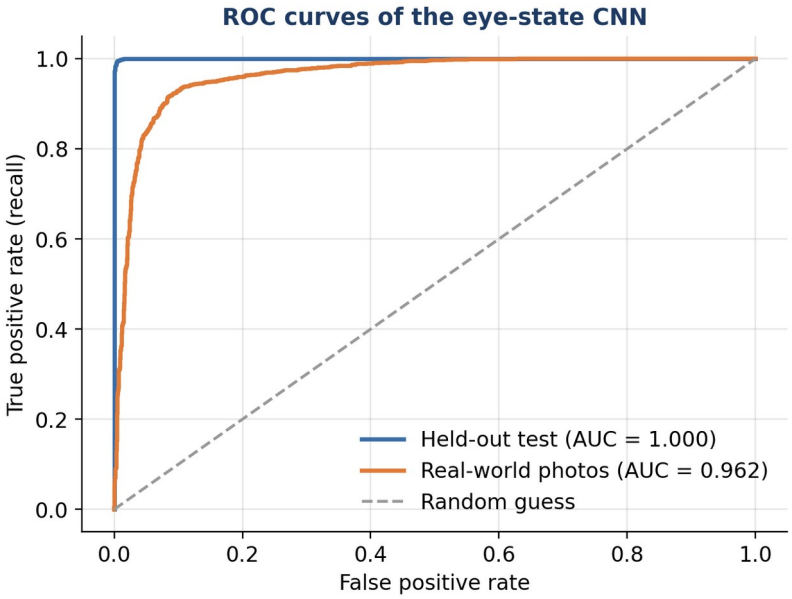


Fig 8: ROC curves on the held-out test set and the real-world set

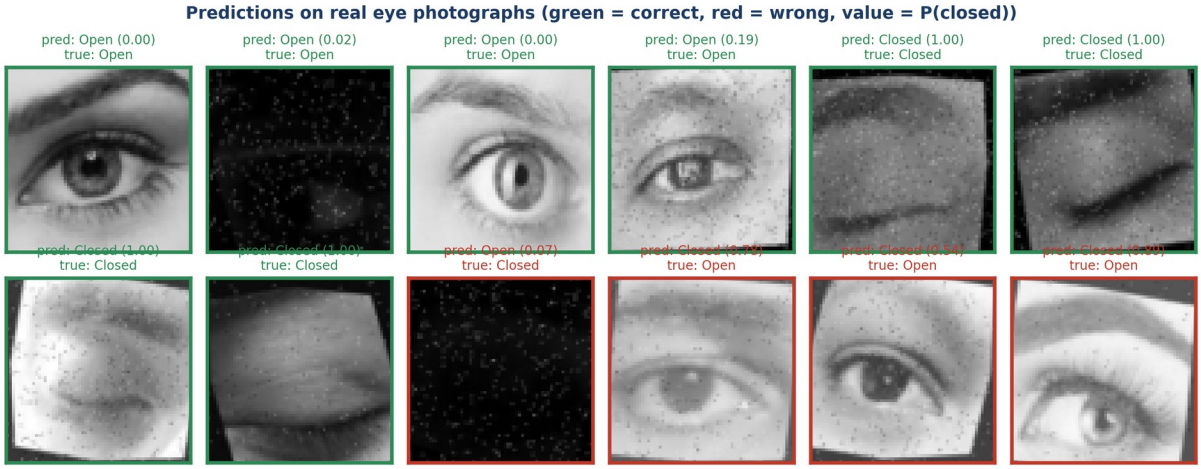


Fig 9: Sample predictions on real eye photographs

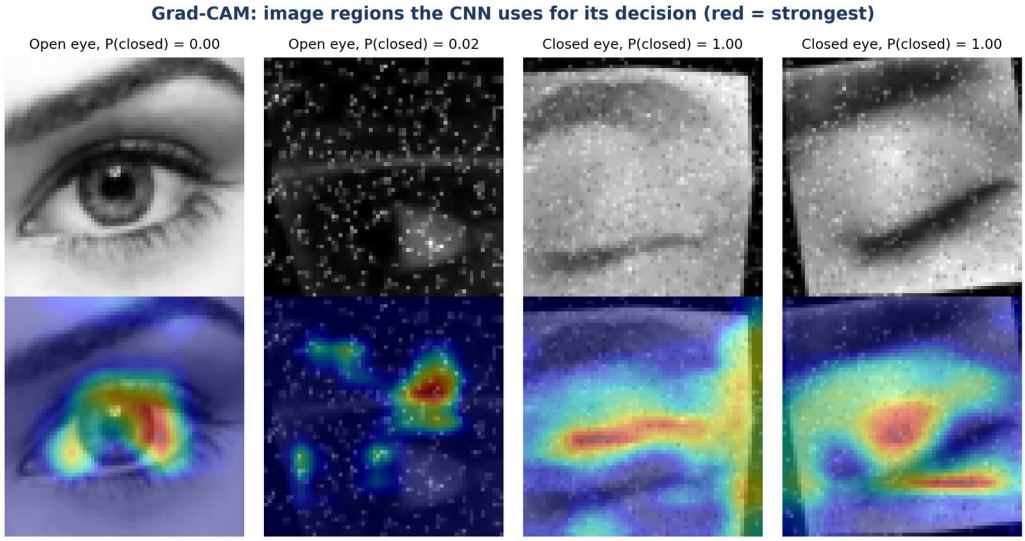


Fig 10: Grad-CAM heatmaps showing where the CNN looks

System-level test on real driver videos

The complete pipeline (eye extraction, CNN and frame counting) was also run on 13,924 frames of real driver videos from the Driver Drowsiness Dataset, which the eye CNN never saw. For each person the share of frames with closed eyes (PERCLOS) was measured in the drowsy clip and the alert clip. The drowsy clip had the higher PERCLOS for 25 of 26 people, with an average of 66.52% against 22.96%.

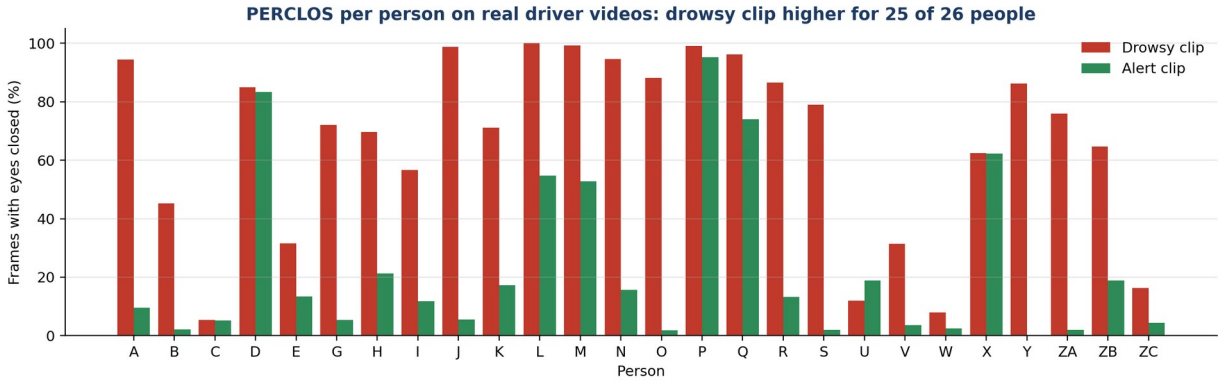


Fig 11: Share of closed-eye frames per person in drowsy and alert videos

Application screenshots

The same driver was uploaded to the Gradio application once with open eyes and once with closed eyes. The face box turns green for an alert driver and red for a drowsy driver, and the two eye regions are marked in yellow.

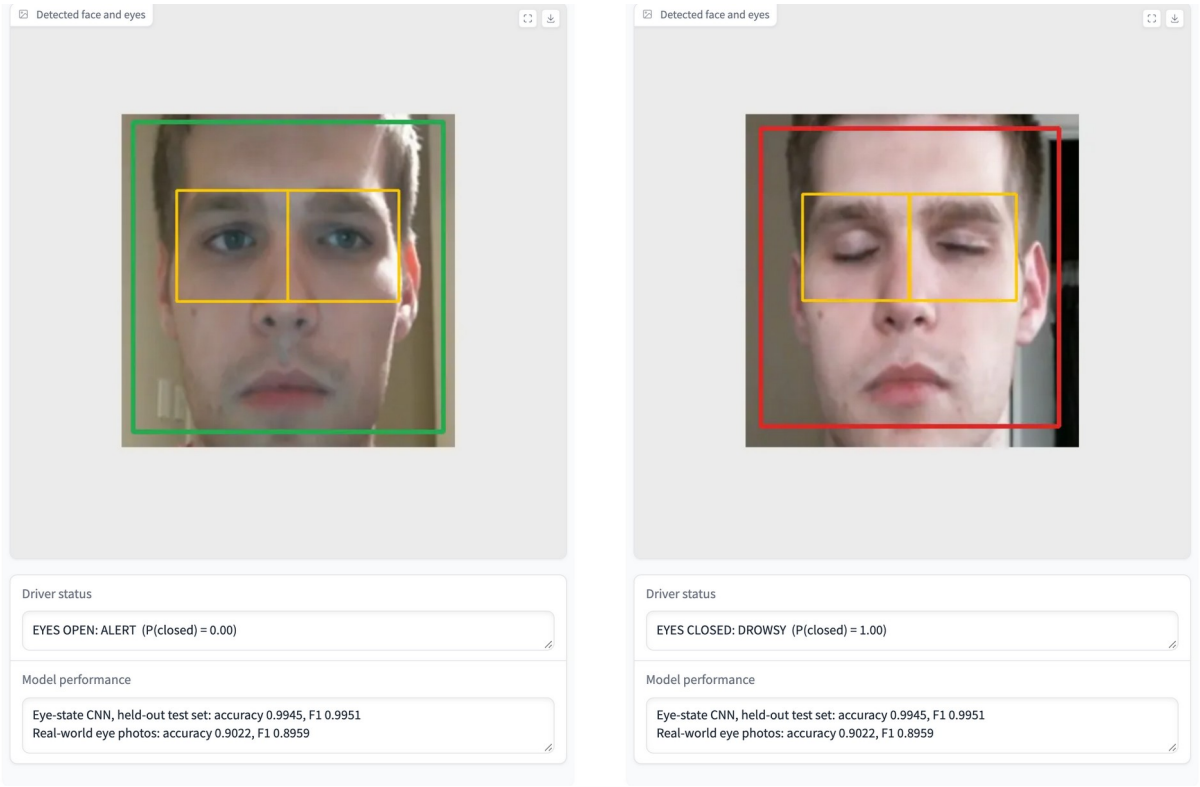


Fig 12: Gradio application output for an alert driver (left) and a drowsy driver (right)

Discussion of the results

- **Held-out performance:** 99.45% accuracy and an AUC of 1.000 on 6,319 eyes from faces not used in training.
- **Real-world generalisation:** on 3,486 real photographs from a different source, accuracy is 90.22% with an AUC of 0.962. The drop shows the domain gap between the training images and real cameras, and is the more realistic figure.
- **Missed closures:** recall on real eyes is 0.9440, so 87 of 1,554 closed eyes were missed. Because the alarm needs 15 closed frames, a single missed frame only delays the alert.
- **False alarms:** 254 of 1,932 open real eyes were called Closed. Requiring 15 consecutive frames filters out most of these isolated errors.
- **What the CNN looks at:** Grad-CAM highlights the eyelid and the iris, which confirms that the decision is based on the eye itself and not on the background.
- **Drowsiness signal:** the drowsy videos had more closed-eye frames for 25 of 26 people, which shows that eye closure is a useful drowsiness signal even though some drowsy clips mostly show open eyes.

The training images are AI-generated faces, and the real-world set is small and contains noisy, rotated images, so these figures show the level of performance and should not be read as a safety certification.

5. Software Used

- **Python 3.9:** the main programming language.

- **TensorFlow 2.20 / Keras:** used to build, train and evaluate the CNN, including augmentation layers.
- **OpenCV 4:** face detection with the Haar cascade, eye cropping and the real-time webcam loop.
- **NumPy:** numerical operations and storage of the prepared datasets.
- **Pillow and PyArrow:** decoding the images stored in the Hugging Face Parquet files.
- **Scikit-learn:** confusion matrix, ROC curve, AUC and the person-wise evaluation.
- **Matplotlib:** training curves, confusion matrices, Grad-CAM and all diagrams.
- **Gradio:** the web interface for uploading a driver image and viewing the result.
- **Hugging Face Datasets:** source of the eye-state, real eye and driver drowsiness data.

6. Conclusion

DriveAlert is a CNN-based system that detects driver drowsiness from a camera by checking, in every frame, whether the eyes are open or closed, and by raising an alarm when they stay closed for 15 frames in a row.

The eye-state CNN reached 99.45% accuracy on held-out eyes and 90.22% on real photographs from a different source. A whole-face classifier was tried first and rejected, because it reached only 69.67% on unseen people. Grad-CAM showed that the network focuses on the eyelids and iris, and the full pipeline found more closed-eye frames in the drowsy videos for 25 of 26 people.

Future improvements could include training on more real driver images, including infrared images for night driving, adding yawning and head nodding as further signs of fatigue, using a facial landmark detector for more precise eye localisation, and testing the system in a real vehicle before any practical use.